

PERÍODO 2 · LÓGICA, ALGORITMOS Y COMPUTACIÓN FÍSICA

8

**Depuración —
buscar el error
como el relojero****Guía 17-8-TIC**

LO QUE TE LLEVAS HOY

Programa en MakeCode con 3 bugs intencionales (entregados por el docente o creados con un compañero) corregidos uno a uno + bitácora de depuración profesional con 3 columnas por cada bug (síntoma observado, hipótesis del problema, prueba que confirma o descarta) + descripción final de 5 líneas sobre cuál bug fue el más difícil y por qué + 1 técnica de depuración aprendida con su nombre

ESTA GUÍA ES DE

Nombre completo

Grupo

Fecha

Autor: Dr. Álvaro Cárdenas Orozco

Metodología MILC · apertura ancestral · pensamiento computacional · triángulo Dussel–estoicismo–Floridi

Apertura: Depuración — buscar el error como el relojero busca el tic-tac

Saber ancestral

En el centro histórico de Cartago, en la carrera 5 cerca de la galería, hubo durante décadas un **taller de relojería** donde un señor mayor de gafas gruesas atendía las piezas que ningún otro mecánico se atrevía a reparar: relojes de pared antiguos heredados de los abuelos, relojes de cuerda traídos de Suiza, despertadores de hojalata de los años 60. El relojero tenía un método que cualquier aprendiz de cualquier oficio podría reconocer como sabiduría profesional: **nunca tocaba dos piezas al mismo tiempo**. Cuando un reloj llegaba averiado, primero **escuchaba**: ¿el tic-tac sonaba bien o cojeaba? ¿había un engranaje que rozaba? ¿la cuerda estaba forzada? Después **formulaba una hipótesis** (“*creo que es el escape*”, decía). Después **ajustaba una sola pieza**, no varias. Después **escuchaba de nuevo**: si el tic-tac se corrigió, había encontrado el error; si no, volvía a la pieza original y formulaba otra hipótesis. **Una hipótesis, una prueba, un resultado**: ese era el método silencioso del relojero, y es exactamente el método profesional de la depuración digital moderna.

Saber contemporáneo

En programación, un **bug** (error de software) es una pieza del programa que no funciona como se esperaba: una variable mal calculada, un if-else con condición invertida, una pausa olvidada, un bucle infinito, una variable que se sobrescribe sin querer. La **depuración** (debugging) es el proceso de encontrar y corregir bugs. La regla profesional famosa, atribuida a Brian Kernighan: **“escribir código es la mitad del oficio; depurar es la otra mitad, y a menudo más difícil que escribir”**. El método profesional de depuración tiene 4 pasos: (1) **Reproducir** el bug consistentemente (saber qué entrada lo provoca). (2) **Formular hipótesis** sobre dónde está el error. (3) **Hacer una sola prueba** (cambiar UN cosa y ver qué pasa). (4) **Confirmar o descartar** la hipótesis. Las **técnicas** más comunes son: *print debugging* (insertar *mostrar variable* para ver qué valor tiene en cada paso), *aislar el problema* (comentar partes del código para encontrar la sección que falla), *usar el simulador* (probar valores extremos).

Pregunta-puente

¿Qué sabía el relojero al cambiar una sola pieza a la vez, que el programador novato olvida cuando toca 5 líneas de código simultáneamente y luego no sabe cuál arregló el problema? ¿Y por qué print debugging es la técnica más vieja y más eficaz del oficio?

Saber y saber hacer

Al terminar podrás: (1) **identificar** los síntomas observables de un bug (qué se esperaba, qué pasó, en qué condiciones); (2) **analizar** el código generando hipótesis posibles sobre dónde está el error; (3) **aplicar** el método de *una hipótesis, una prueba, un resultado* para encontrar el bug sin tocar otras partes del código; (4) **evaluar** tu propio proceso de depuración nombrando al menos 1 técnica aprendida y reflexionando sobre cuál bug fue el más difícil.

Autor	Dr. Álvaro Cárdenas Orozco
DBA	Pensamiento computacional aplicado a sistemas de control con sensores y actuadores (MEN, T&I)
Referentes	Pensamiento computacional · MakeCode / micro:bit · Logos como razón universal
Producto final	Programa en MakeCode con 3 bugs intencionales (entregados por el docente o creados con un compañero) corregidos uno a uno + bitácora de depuración profesional con 3 columnas por cada bug (síntoma observado, hipótesis del problema, prueba que confirma o descarta) + descripción final de 5 líneas sobre cuál bug fue el más difícil y por qué + 1 técnica de depuración aprendida con su nombre

→ Antes de empezar, mira el viaje completo. En las sesiones 4-6 aprendiste a programar con sensores, actuadores y umbrales. Hoy aprendes a reparar código cuando no hace lo que esperas. Sin habilidad de depuración, cualquier programa que crezca un poco se convierte en problema sin salida.

Ruta MILC: así vamos a aprender

1

Escucha
Observo, escucho
y pregunto.

2

Entender
Sistematizo
y ordeno.

3

Hacer
Construyo y aplico.

4

Cierre
Evalúo y reflexiono.

→ Antes de teorizar, vas a hacer un ejercicio del relojero: tu docente proyecta un programa MakeCode con 1 bug evidente (“debería mostrar el ícono triste pero muestra el feliz”). Sin tocar el código todavía, anota tus 3 hipótesis posibles sobre dónde está el error.

Estación 1 de 3 · Escucha

Escena para escuchar

Actividad 1 · IDENTIFICA — Ejercicio del relojero (15 min · individual). Tu docente proyecta o entrega un programa MakeCode con 1 bug: “debería mostrar ícono triste cuando la temperatura es menor a 18 grados, pero muestra ícono feliz”. Sin tocar el código todavía, mira el bloque if-else y anota 3 hipótesis posibles: (1) ¿la condición está invertida (mayor en lugar de menor)?, (2) ¿el ícono está cambiado de rama?, (3) ¿la variable de temperatura está mal leída? Ordena tus hipótesis de la más probable a la menos probable.

MARCA CUANDO LO HAYAS HECHO

- ☐ Anoté 3 hipótesis distintas sobre el bug.
- ☐ Las ordené por probabilidad.
- ☐ No toqué el código todavía.

Lo que va al cuaderno (Actividad 1 · IDENTIFICA). Título: «Actividad 1 — Hipótesis del bug». **Formato:** 3 hipótesis numeradas + orden de probabilidad. **Sabes que terminaste cuando** tienes el plan de qué probar primero.

→ Ya tienes hipótesis. Ahora vas a entender la **anatomía profesional de la depuración**: los 4 pasos del método, las técnicas más comunes, los 5 errores típicos del depurador novato.

Estación 2 de 3 · Entender

Lo que vas a entender

Tus hipótesis son el primer paso del método. Ahora necesitas la **anatomía profesional** de los 4 pasos de depuración, las técnicas usadas en la industria y los errores que delatan al depurador novato.

- 1 Los **4 pasos del método profesional** se descomponen así: (1) **Reproducir**: identifica exactamente qué entrada produce el bug. ¿Pasa siempre o solo a veces?, ¿con qué valores?. Sin reproducir, no se puede depurar. (2) **Hipótesis**: formula 2-3 explicaciones posibles ordenadas por probabilidad. Lo más probable primero. (3) **Una prueba**: cambia UNA sola cosa para verificar UNA hipótesis. Si cambias varias, no sabrás cuál arregló el problema (o cuál lo empeoró). (4) **Confirmar o descartar**: si la hipótesis era correcta, el bug se corrigió. Si no, vuelve al código original y prueba la siguiente.
- 2 Las **3 técnicas más comunes** de depuración son: (a) **Print debugging**: insertar *mostrar variable* o *mostrar cadena* en puntos del código para ver qué valor tiene cada variable en cada momento. Es la técnica más vieja y la más eficaz. (b) **Aislar el problema**: comentar (deshabilitar) partes del código para encontrar la sección que falla. En MakeCode se hace clic derecho sobre un bloque y *deshabilitar*. (c) **Probar valores extremos**: usar entradas extremas (0, número máximo, vacío) para forzar el bug a manifestarse.
- 3 Lo esencial cabe en una pregunta: *¿qué cambio me dará información sobre el bug?*. Si haces un cambio que no te da información (mover bloques al azar, cambiar varios valores juntos), no estás depurando: estás esperando que el bug se solucione por azar. La phronesis del depurador consiste en **convertir cada prueba en aprendizaje**, no en intento ciego.
- 4 Algoritmo: (1) reproducir el bug. (2) listar 2-3 hipótesis. (3) elegir la más probable. (4) hacer una prueba específica (insertar print, deshabilitar bloque, cambiar 1 valor). (5) observar resultado. (6) si confirmó hipótesis, corregir; si no, volver al código original y probar siguiente hipótesis. (7) documentar todo en bitácora.

Anatomía de la depuración

Reproducir: con qué entrada falla.

Hipótesis: 2-3 explicaciones ordenadas.

Una prueba: cambiar una sola cosa.

Confirmar/descartar: interpretar resultado.

Técnicas: print debugging / aislar / valores extremos.

Errores comunes y su corrección

(1) **Cambiar varias cosas a la vez:** imposible saber cuál arregló o empeoró. (2) **No reproducir el bug:** si el bug pasa solo a veces y no sabes cuándo, no puedes depurarlo. (3) **Asumir sin probar:** “*debe ser la variable*” sin verificarlo. (4) **No documentar el proceso:** olvidas qué probaste y caes en bucle de pruebas repetidas. (5) **Frustrarse y romper más:** agregar cambios al azar cuando el bug no cede empeora la situación.

Lo que va al cuaderno (Actividad 2 · ANALIZA). Título: «Actividad 2 — Anatomía de la depuración».

Formato: (a) los 4 pasos del método; (b) las 3 técnicas con su uso; (c) los 5 errores comunes con marca.

Sabes que terminaste cuando entiendes por qué *una hipótesis, una prueba* es el método y no *tocar varias cosas a la vez*.

→ Tienes el método. Toca aplicarlo: tomas un programa con 3 bugs intencionales (el docente lo entrega o lo arman entre compañeros intercambiando código con errores plantados) y los corriges uno a uno con bitácora profesional.

Estación 3 de 3 · Hacer

Cómo vas a construir tu producto

Actividad 3 · APLICA + EVALÚA — 3 bugs corregidos con bitácora profesional (40 min · individual o parejas). Hora de aplicar. El docente entrega un programa con 3 bugs intencionales (o intercambian códigos entre compañeros con errores plantados), y tú los corriges uno a uno documentando todo el proceso.

1

Los 3 bugs sugeridos para esta sesión (el docente puede usar estos o crear otros): (1) **Bug de condición invertida:** el if dice si `temperatura > 18` cuando debería ser `<` 18. (2) **Bug de variable no inicializada:** una variable se usa sin haber sido establecida primero. (3) **Bug de pausa olvidada:** una animación va tan rápido que solo se ve el último frame porque falta el bloque pausa. Cada uno se corrige con método: reproducir, hipotetizar, probar.

2 Sigue el patrón **“bitácora completa por cada bug”**: la bitácora tiene 3 columnas: (a) **Síntoma**: qué hace mal el programa (*“cuando hace 30 grados muestra frío”*). (b) **Hipótesis**: dónde crees que está el error (*“creo que la condición está invertida”*). (c) **Prueba y resultado**: qué probaste y qué pasó (*“cambié > por < y ahora muestra correctamente; hipótesis confirmada”*). Si una hipótesis se descarta, anótalo y pasa a la siguiente.

3 Las técnicas para aplicar: (1) usa *mostrar variable* para verificar valores intermedios. (2) deshabilita bloques temporalmente para aislar el problema. (3) prueba con valores extremos (temperatura muy alta, muy baja). Documenta cuál técnica usaste con cada bug en la reflexión final.

4 Algoritmo: (1) recibir programa con bugs. (2) probar el programa para identificar qué falla. (3) para el bug 1: hipótesis → prueba → confirmar/descartar → corregir. (4) repetir para bug 2 y 3. (5) reflexión final: ¿cuál bug fue más difícil?, ¿qué técnica usaste más?, ¿qué aprendiste sobre tu propio proceso?

Checklist antes de entregar

- ☐ 3 bugs identificados con su síntoma.
- ☐ Para cada bug, al menos 1 hipótesis formulada.
- ☐ Cada prueba cambia UNA sola cosa.
- ☐ Bitácora con 3 columnas por bug (síntoma, hipótesis, prueba/resultado).
- ☐ 3 bugs corregidos.
- ☐ Reflexión final con bug más difícil + técnica aprendida.

Plantilla de trabajo

Bug 1. Síntoma + Hipótesis + Prueba/Resultado.

Bug 2. Síntoma + Hipótesis + Prueba/Resultado.

Bug 3. Síntoma + Hipótesis + Prueba/Resultado.

Reflexión. Cuál fue más difícil + por qué + técnica usada.

Lo que va al cuaderno (Actividad 3 · APLICA + EVALÚA). Título: «Actividad 3 — 3 bugs corregidos».

Formato: (a) bitácora 3 filas × 3 columnas; (b) reflexión final de 5 líneas; (c) técnica de depuración aprendida con su nombre. **Sabes que terminaste cuando** podrías delegar la depuración de un programa similar a un compañero con tu bitácora como guía.

→ Lo que sigue resume qué entregas hoy: 3 bugs corregidos + bitácora con 3 columnas por bug + reflexión sobre el más difícil + nombre de 1 técnica aprendida. El criterio principal: que el proceso de depuración esté documentado lo suficientemente bien para que un compañero pueda reproducirlo.

Producto de la sesión

3 bugs corregidos + bitácora 3 columnas + reflexión + 1 técnica.

Criterios de calidad

(1) 3 bugs identificados y corregidos. (2) Bitácora con 3 columnas por bug. (3) Una hipótesis por prueba (no múltiples cambios). (4) Reflexión sobre el bug más difícil. (5) 1 técnica de depuración nombrada. (6) Programa final funcional.

→ Antes de cerrar, mira la depuración desde las cinco dimensiones humanas. Documentar el proceso es respeto por quien herede el código; cambiar todo sin pensar es desprecio por el oficio.

Evaluación liberadora · cinco dimensiones

Sentido de la evaluación

Evaluar no es solo revisar una nota. Es reconocer qué aprendí, qué cambió en mí, qué emociones manejé, cómo me relaciono con la ciudad y la comunidad, y qué le dejaré a quien viene detrás.

Mi reflexión liberadora en cinco dimensiones. Completa cada frase iniciada:

Dimensión	Mi respuesta (frame starter)	Algo que descubrí
1. Personal	Lo que más cambió en mí fue _____	_____
2. Emocional	La emoción más fuerte fue _____ y la regulé _____	_____
3. Ciudadana	En democracia esto importa porque _____	_____
4. Local (Cartago)	En mi barrio sería útil para _____	_____
5. Intergeneracional	A mi abuela/abuelo le diría _____	_____

Para la próxima sustentación. Vuelve a esta tabla en seis meses. Verás que las respuestas cambian — no porque lo aprendido cambie, sino porque tú cambias. La evaluación liberadora es una conversación contigo a través del tiempo.

→ Y para terminar, tres voces sobre el oficio del error. Dussel sobre los bugs que afectan a quienes confiaron en el sistema, Marco Aurelio sobre la disciplina de una hipótesis a la vez, Floridi sobre la depuración como ética profesional contemporánea.

Triángulo de pensamiento · cierre filosófico

Dussel · lente del nosotros

Un bug ignorado afecta a quien usa el sistema; documentar la depuración es respeto por la cadena de usuarios futuros.

Cuando depuras un código sin documentar, solo tú entiendes qué pasó. Si otro programador hereda el código (o tú mismo lo vuelves a abrir en 6 meses), no podrá entender por qué se hicieron ciertas decisiones. La filosofía de la liberación pide documentar el proceso de depuración: la bitácora no es trámite, es **respeto por la cadena de personas que tocarán el código después**.

¿Mi bitácora permitiría que otro programador entienda qué hice y por qué, o solo yo sé qué pasó?

Estoicismo · Marco Aurelio · cuidado interior

Una hipótesis a la vez es disciplina; tocar varias piezas simultáneas es vanidad disfrazada de eficiencia.

Es tentador cambiar varias cosas al mismo tiempo para “arreglar más rápido”. La sabiduría estoica recomienda lo opuesto: *una hipótesis, una prueba, un resultado*. Esa paciencia parece lenta pero es la única que produce aprendizaje. El depurador que cambia varias cosas a la vez puede arreglar el bug por azar, pero no aprende qué pasó.

¿Estoy probando una hipótesis a la vez, o cambiando varias cosas esperando que algo funcione?

Floridi · lente de la infoesfera

La depuración rigurosa es la nueva ética profesional del oficio digital en la era del software ubicuo.

En la era de la información, el software opera en todas partes: bancos, hospitales, transporte. Un bug en cualquiera de esos sistemas puede tener consecuencias graves para personas reales. La ética digital exige depuración rigurosa: no “*parece funcionar*”, sino “*probado con método, documentado, corregido*”. Tu hoja de hoy te entrena en esa práctica profesional que rige toda industria del siglo XXI.

¿Mi depuración tiene el rigor que tendría si este código fuera para un sistema crítico, o me conformo con que parezca funcionar?

Nota para el docente. Las tres formulaciones del triángulo son la síntesis del autor de la guía, no citas textuales de los pensadores. Si vas a citarlos en clase, remite a la obra original.

Mi autoevaluación · marca una casilla por fila

Criterio	Lo logré	En proceso	Necesito apoyo
Comprensión del tema	☐ Explico las ideas con mis palabras.	☐ Reconozco las ideas, falta precisión.	☐ Necesito repasar lo básico.
Pensamiento computacional	☐ Usé los pilares al armar mi producto.	☐ Usé algunos pilares.	☐ Necesito releer la estación 2.
Aplicación y producto	☐ Mi producto cumple los criterios.	☐ Está armado, con ajustes pendientes.	☐ Aún está en construcción.
Reflexión 5 dimensiones	☐ Respondí cada una con honestidad.	☐ Respondí algunas con detalle.	☐ Mis respuestas son muy generales.
Triángulo de pensamiento	☐ Conecté las 3 voces con mi trabajo.	☐ Conecté al menos una.	☐ No las relaciono con mi proceso.

Hoy entendí que:

Mi compromiso para la próxima sesión es:

Cita de despedida · Marco Aurelio

“El obstáculo en el camino se vuelve el camino. Lo que estorba al camino se vuelve camino.”

Cada error de hoy, bien observado, se convierte en pieza del camino que pisarás mañana.

Nombre completo:

Fecha: _____ Curso/grupo: _____

Mi firma: _____

Para la próxima sesión. Llega con tu bitácora de los 3 bugs corregidos. En la sesión 8 vas a aprender a construir **alertas con lógica compuesta** usando múltiples sensores combinados con AND/OR/NOT. La depuración de hoy es la garantía: cuando tu lógica compuesta falle, sabrás cómo encontrar el error con método.